UET NOWSHERA | ELECTRICAL ENGINEERING

LAB REPORT 09

INTRODUCTION TO FUNCTIONS IN C++

1. OBJECTIVE

- To understand the concept and purpose of functions in C++.
- To learn how to declare function prototypes and define function bodies.
- To pass parameters to functions and handle return values.
- To create modular, reusable code blocks instead of writing everything in main().

2. THEORETICAL BACKGROUND

Introduction to Functions. Functions let you chop up a long program into named sections so that the sections can be reused throughout the program. A function has a name identifier, accepts parameters, and returns a result. C++ functions can accept multiple parameters separated by commas. In general, C++ does not care in what order you put your functions, so long as the function name is known to the compiler before it is called.

Function Structure. When we actually call functions, the code inside them executes, and at the end, it transfers control back to the program from where the function was called. Function is differentiated from a simple variable by the parenthesis just after the name, e.g., main(). Anything inside the brackets are the parameters passed to the function.

Return Values. When we define our own functions, that function should return an appropriate value according to the data type which is given before the name of the function. C++ assumes that every function will return a value using the return statement

unless the function is explicitly declared as void, which indicates no return value is required.

Prototype of Function. The prototype of a function is a way of describing the parameters (also called arguments) and parameter types with which a legal call to the function can be made. It contains the name of the function, its parameters, their type, and the return value. Syntax: returnType functionName(parameters);

3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

4. EXPERIMENTAL TASKS

Task 1: Function Introduction and Declaration

A function named 'greet' was declared to take no parameters and return a void type. Inside the function, a custom greeting message is printed. This function is then called from within the main() function to execute the display.

```cpp
#include <iostream>
using namespace std;

// Function prototype and definition
void greet() {
    cout << "Welcome to the C++ Programming Lab!" << endl;
}

int main() {
    // Function call
    greet();
    return 0;
}
```

Task 2: Basic Function Declaration and Call

Similar to the previous task, a void function named 'sayHello' was declared. It prints the standard 'Hello, World!' message. The function is invoked inside the main() program block, demonstrating a basic function call structure.

```cpp
#include <iostream>
using namespace std;

// Function definition
void sayHello() {
    cout << "Hello, World!" << endl;
}

int main() {
    // Function call to execute the greeting
    sayHello();
    return 0;
}
```

Task 3: Array Sum and Average using Function

An integer array of size 10 is declared to store user inputs. A separate function 'calculateSum' is defined, which takes the array and its size as parameters, iterates through the array using a for loop, and returns the total sum. In the main() function, the returned sum is then used to calculate and display the average.

```cpp
#include <iostream>
using namespace std;

// Function to calculate the sum of an array
int calculateSum(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}

int main() {
    int numbers[10];

    // Take 10 integer inputs from the user
    cout << "Enter 10 integer values:\n";
    for (int i = 0; i < 10; i++) {
        cout << "Value " << i + 1 << ": ";
        cin >> numbers[i];
    }

    // Call function to get sum
    int totalSum = calculateSum(numbers, 10);

    // Calculate average
    double average = static_cast<double>(totalSum) / 10.0;

    // Display results
    cout << "\nSum of all elements: " << totalSum << endl;
    cout << "Average of elements: " << average << endl;

    return 0;
}
```

5. CONCLUSION

This lab introduced the fundamental concepts of functions in C++, including function prototypes, definitions, and calling mechanisms. It demonstrated how to compartmentalize code into reusable blocks, handle functions without parameters (using void), and pass arrays as parameters to perform computational tasks like finding the sum and average. These foundational concepts are crucial for writing modular, clean, and easily maintainable programs in subsequent software development tasks.